1 An Overview of C++

Chapter Objectives

1.1 What is Object-Oriented Programming

1.2 Two versions of C++

1.3 C++ console I/O

1.4 C++ comments

1.5 Classes: A first look

1.6 Some differences between C and C++

1.7 Introducing function overloading

1.8 C++ keywords

C++ is an enhanced version of the C language. C++ includes everything that is part of C and adds support for object-oriented programming (OOP for short). In addition, C++ contains many improvements and features that simply make it a "better C," independent of object-oriented programming. With very few, very minor exceptions, C++ is a superset of C. While everything that you know about the C language is fully applicable to C++, understanding its enhanced features will still require a significant investment of time and effort on your part. However, the rewards of programming in C++ will more than justify the effort you put forth.
The purpose of this chapter is to introduce you to several of the most important features of C++. As you know, the elements of a computer language do not exist in a void, separate from one another. Instead, they work together to form the complete language. This interrelatedness is especially pronounced in C++. In fact, it is difficult to discuss one aspect of C++ in isolation because the features of C++ are highly integrated. To help overcome this problem, this chapter provides a brief overview of several C++ features. This overview will enable you to understand the examples discussed later in this book. Keep in mind that most topics will be more thoroughly explored in later chapters.
Since C++ was invented to support object-oriented programming, this chapter begins with a description of OOP. As you will see, many features of C++ are related to OOP in one way or another. In fact, the theory of OOP permeates C++. However, it is important to understand that C++ can be used to write programs that are and are *not* object oriented. How you use C++ is completely up to you.
At the time of this writing, the standardization of C++ is being finalized. For this reason, this chapter describes some important differences between versions of C++ that have been in common use during the past several years and the new Standard C++. Since this books teaches Standard C++, this material is especially important if you are using an older compiler.
In addition to introducing several important C++ features, this chapter also discusses some differences between C and C++ programming styles. There are several aspects of C++ that allow greater flexibility in the way that you write programs. While some of these features have little or nothing to do with object-oriented programming, they are found in most C++ programs, so it is appropriate to discuss them early in this book.
Before you begin, a few general comments about the nature and form of C++ are in order. First, for the most part, C++ programs physically look like C programs. Like a C program, a C++ program begins execution at **main()**. To include command-line arguments, C++ uses the same **argc**, **argv** convention that C uses. Although C++ defines its own, object-oriented library, it also supports all the functions in the C standard library. C++ uses the same control structures as C. C++ includes all of the built-in data types defined by C.
This book assumes that you already know the C programming language. In other words, you must be able to program in C before you can learn to program in C++ by using this book. If you don;t know C, a good starting place is my book *Teach Yourself C, Third Edition* (Berkeley: Osborne/McGraw-Hill, 1997). It applies the same systematic approach used in this book and thoroughly covers the entire C language.
Note: *This book assumes that you know how to compile and execute a program using your C++ compiler. If you don't, you will need to refer to your compiler's instructions. (Because of the differences between compilers, it is impossible to give compilation instructions for each in this book.) Since programming is beast learned by doing, you are strongly urged to enter, compile and run the examples in the book in the order in which they are presented.*

1.1 WHAT IS OBJECT-ORIENTED PROGRAMMING?

Object-oriented programming is a powerful way to approach the task of programming. Since its early beginnings, programming has been governed by various methodologies. At each critical point in the evolution of programming, a new approach was created to help the programmer

handle increasingly complex programs. The first programs were created by toggling switches on the front panel of the computer. Obviously, this approach is suitable for only the smallest programs. Next, assembly language was invented, which allowed longer programs to be written. The next advance happened in the 1950s when first high-level language (FORTRAN) was invented.

By using a high-level language, a programmer was able to write programs that were several thousand lines long. However, the method of programming used early on was an ad hoc, anything-goes approach. While this is fine for relatively short programs, it yields unreadable (and unmanageable) "spaghetti code" when applied to larger programs. The elimination of spaghetti code became feasible with the invention of *structured programming languages* in the 1960s. These languages include Algol and Pascal. In loose terms, C is a structured language, and most likely the type of programming you have been doing would be called structured programming. structured programming relies on well-defined control structures, code blocks, the absence (or at least minimal use) of the GOTO, and stand-alone subroutines that support recursion and local variables. The essence of structured programming is the reduction of a program into its constituent elements. Using structured programming, the average programmer can create and maintain programs that are up to 50,000 lines long.

Although structured programming has yielded excellent results when applied to moderately complex programs, even it fails at some point, after a program reaches a certain size. To allow more complex programs to be written, a new approach to the job of programming was needed. Towards this end, object-oriented programming was invented. OOP takes the best of the ideas embodied in structured programming and combines them with powerful new concepts that allow you to organize your programs more effectively. Object-oriented programming encourages you to decompose a problem into its constituent parts. Each component becomes a self-contained object that contains its own instructions and data that relate to that object. In this way, complexity is reduced and the programmer can manage larger programs.

All OOP languages, including C++, share three common defining traits: encapsulation, polymorphism, and inheritance. Let's look at these concepts now.

ENCAPSULATION

Encapsulation is the mechanism that binds together code and the data it manipulates, and keeps both safe from outside interference and misuse. In an object-oriented language, code and data can be combined in such a way that a self-contained "black box" is created. When code and data are linked together in this fashion, an *object* is created. In other words, an object is the device that supports encapsulation.

Within an object, code, data, or both may be *private* to that object or *public*. Private code or data is known to and accessible only by another part of the object. That is, private code or data cannot be accessed by a piece of the program that exists outside the object. When code or data is public, other parts of your program can access it even though it is defined within an object. Typically, the public parts of an object are used to provide a controlled interface to the private elements of the object.

For all intents and purposes, an object is a variable of a user-defined type. It may seem strange that an object that links both code and data can be thought of as a variable. However, in object-oriented programming, this is precisely the case. Each time you define a new type of object, you are creating a new data type. Each specific instance of this data type is a compound variable.

POLYMORPHISM

Polymorphism (from the Greek, meaning "many forms") is the quality that allows one name to be used for two or more related but technically different purposes. As it relates to OOP, polymorphism allows one name to specify a general class of actions. Within a

general class of actions, the specific action to be applied is determined by the type of data. For example, in C, which does not significantly support polymorphism, the absolute value action requires three distinct function names: **abs()**, **labs()**, and **fabs()**. These functions compute and return the absolute value of an integer, a long integer, and a floating-point value, respectively. However, in C++, which supports polymorphism, each function can be called by the same name, such as **abs()**. (One way this can be accomplished is shown later in this chapter.) The type of data used to call the function determines which specific version of the function is actually executed. As you will see, in C++, it is possible to use one function name for many different purposes. This is called *function overloading*.

More generally, the concept of polymorphism is characterized by the idea of "one interface, multiple methods," which means using a generic interface for a group of related activities. The advantage of polymorphism is that it helps to reduce complexity by allowing one interface to specify a *general class of action*. It is the compiler's job to select the *specific action* as it applies to each situation. You, the programmer, don't need to do this selection manually. You need only remember and utilize the general interface. As the example in the preceding paragraph illustrates, having three names for the absolute value function instead of just one makes the general activity of obtaining the absolute value of a number more complex than it actually is.

Polymorphism can be applied to operators, too. Virtually all programming languages contain a limited application of polymorphism as it relates to the arithmetic operators. For example, in C, the + sign is used to add integers, long integers, characters, and floating-point values. In these cases, the compiler automatically knows which type of arithmetic to apply. In C++, you can extend this concept to other types of data that you define. This type of polymorphism is called *operator overloading*.

The key point to remember about polymorphism is that it allows you to handle greater complexity by allowing the creation of standard interfaces to related activities.

INHERITANCE

Inheritance is the process by which one object can acquire the properties of another. More specifically, an object can inherit a general set of properties to which it can add those features that are specific only to itself. Inheritance is important because it allows an object to support the concept of *hierarchical classification*. Most information is made manageable by hierarchical classification. For example, think about the description of a house. A house is apart of the general class called **building**. In turn, **building** is part of the more general class **structure**, which is part of the even more general class of objects that we call **man-made**. In each case, the child class inherits all those qualities associated with the parent and adds to them its own defining characteristics. Without the use of ordered classifications, each object would have to define all characteristics that relate to it explicitly. However, through inheritance, it is possible to describe an object by stating what general class (or classes) it belongs to along with those specific traits that make it unique. As you will see, inheritance plays a very important role in OOP.

EXAMPLES

1. Encapsulation is not entirely new to OOP. To a degree, encapsulation can be achieved when using the C language. For example, when you use a library function, you are, in effect, using a black-box routine, the internals of which you cannot alter or affect (except, perhaps, through malicious actions). Consider the **fopen()** function. When it is used to open a file, several internal variables are created and initialized. As far as your program is concerned, these variables are hidden and not accessible. However, C++ provides a much more secure approach to encapsulation.

2. In the real world, examples of polymorphism are quite common. For example, consider the steering wheel on your car. It works the same whether your car uses power steering, rack-and-pinion steering, or standard, manual steering. The point is that the interface (the steering wheel) is the same no matter what type of actual steering mechanism (method) is used.

3. Inheritance of properties and the more general concept of classification are fundamental to the way of knowledge is organized. For example, celery is a member of the **vegetable** class, which is part of the **plant** class. In turn, plants are living organisms, and so forth. Without hierarchical classification, systems of knowledge would not be possible.

EXERCISE

1. Think about the way that classification and polymorphism play an important role in our day-to-day lives.

1.2 TWO VERSIONS OF C++

At the time of this writing, C++ is in the midst of a transformation. As explained in the preface to this book, C++ has been undergoing the process of standardization for the past several years. The goal has been to create a stable, standardized, feature-rich language that will suit the needs of programmers well into the next century. As a result, there are really two versions of C++. The first is the traditional version that is based upon Bjarne Stroustrup's original designs. This is the version of C++ that has been used by programmers for the past decade. The second is the new Standard C++, which was created by Stroustrup and the ANSI/ISO standardization committee. While these two versions of C++ are very similar at their core, Standard C++ contains several enhancements not found in traditional C++. Thus, Standard C++ is essentially a superset of traditional C++.

This book teaches Standard C++. This is the version of C++ defined by the ANSI/ISO standardization committee, and it is the version implemented by all modern C++ compilers. The code in this book reflects the contemporary coding style and practices as encouraged by Standard C++. This means that what you learn in this book will be applicable today as well as tomorrow. Put directly, Standard C++ is the future. And, since Standard C++ encompasses all features found in earlier versions of C++, what you learn in this book will enable you to work in all C++ programming environments.

However, if you are using an older compiler, it might not accept all of the programs in this book. Here's why: During the process of standardization, the ANSI/ISO committee added any new features to the language. As these features were defined, they were implemented by compiler developers. Of course, there is always a lag time between the addition of a new feature to the language and the availability of the feature in commercial compilers. Since features were added to C++ over a period of years, an older compiler might not support one or more of them. This is important because two recent additions to the C++ language affect every program that you will write-even the simplest. If you are using an older compiler that does not accept these new features, don't worry. There is an easy workaround, which is described in the following paragraphs.

The differences between old-style and modern code involve two new features: new-style headers and the **namespace** statement. To demonstrate these differences we will begin by looking at two versions of a minimal, do-nothing C++ program. The first version, shown here, reflects the way C++ programs were written until recently. (That is, it uses old-style coding.)

```
/*
    A traditional-style C++ program.
```

```
*/

#include <iostream.h>

int main()
{
    /* program code */
    return 0;
}
```

Since C++ is build on C, this skeleton should be largely familiar, but pay special attention to the **#include** statement. This statement includes the file **iostream.h**, which provides support for C++'s I/O system. (It is to C++ what **stdio.h** is to C.)
Here is the second version of the skeleton, which uses the modern style:

```
/*
    A modern-style C++ program that uses
    the new-style headers and a namespace.
*/
#include <iostream>
using namespace std;

int main()
{
    /* program code */
    return 0;
}
```

Notice the two lines in this program immediately after the first comment; this is where the changes occur. First, in the **#include** statement, there is no **.h** after the name **iostream**. And second, the next line, specifying a namespaces will be examined in detail later in this book, a brief overview is in order now.

THE NEW C++ HEADERS

As you know from your C programming experience, when you use a library function in a program, you must include its header file. This is done using the **#include** statement. For example, in C, to include the header file for the I/O functions, you include **stdio.h** with a statement like this:

```
#include <stdio.h>
```

Here **stdio.h** is the name of the file used by the I/O functions, and the preceding statement cause that file to be included in your program. The key point is that the **#include** statement *includes a file.*
When C++ was first invented and for several years after that, it used the same style of headers as did C. In fact, Standard C++ still supports C-style headers for header files that you create and for backward compatibility. However, Standard C++ has introduced a new kind of header that is used by the Standard C++ library. The new-style headers *do not* specify filenames. Instead, they simply specify standard identifiers that might be mapped o files by the compiler, but they need not be. The new-style C++ headers are abstractions that simply guarantee that the appropriate prototypes and definitions required by the C++ library have been declared.
Since the new-style header is not a filename, it does not have a **.h** extension. Such a header consists solely of the header name contained between angle brackets. For example, here are some of the new-style headers supported by Standard C++:

```
iostream>
<fstream>
<vector>
<string>
```

The new-style headers are included using the **#include** statement. The only difference is that the new-style headers do not necessarily represent filenames.
Because C++ includes the entire C function library, it still supports the standard C-style header files associated with that library. That is, header files such as **stdio.h** and **ctype.h** are still available. However, Standard C++ also defines new-style headers that you can use in place of these header files. The C++ versions of the standard C headers simply add a **c** prefix to the filename and drop the **.h**. For example, the new style C++ header for **math.h** is **<cmath>**, and the one for **string.h** is **<cstring>**. Although it is currently permissible to include a C-style header file when using C library functions, this approach is deprecated by Standard C++. (That is, it is not recommended.) For this reason, this book will use new-style C++ headers in all **#include** statements. If your compiler does not support new-style headers the C function library, simply substitute the old-style, C-like headers.
Since the new-style header is a recent addition to C++, you will still find many, many older programs that don't use it. These programs instead use C-style headers, in which a filename is specified. As the old-style skeletal program shows, the traditional way to include the I/O header is as shown here:

```
#include <iostream.h>
```

This causes the file **iostream.h** to be included in your program. In general, an old-style header will use the same name as its corresponding new-style header with a **.h** appended.
As of this writing, all C++ compilers support the old-style headers. However, the old style headers have been declared obsolete, and their use in new programs is not recommended. This is why they are not used in this book.
Remember: *While still common in existing C++ code, old-style headers are obsolete.*

NAMESPACES

When you include a new-style header in your program the contents of that header are contained in the **std** namespace. A *namespace* is simply a declarative region. The purpose of a namespace is to localize the names of identifiers to avoid name collisions. Traditionally, the names of library functions and other such items were simply placed into the global namespace (as they are in C). However, the contents of new-style headers are placed in the **std** namespace. We will look closely at namespaces later in this book. For now, you don't need to worry about them because you can use the statement

```
using namespace std;
```

to bring the **std** namespace into visibility (i.e., to put **std** into the global namespace). After this statement has been compiled, there is no difference between working with an old-style header and a new-style one.

WORKING WITH AN OLD COMPILER

As mentioned, both namespaces and the new-style are recent additions to the C++ language. While virtually all new C++ compilers support these features, older compilers might not. If you have one of these older compilers, it will report one or more errors when it tries to compile the first two lines of the sample programs in this book. If this is the case, there is an easy workaround: simply use an old-style header and delete the **namespace** statement. That is, just replace

```
#include <iostream>
using namespace std;
```

with

```
#include <iostream.h>
```

This change transforms a modern program into a traditional-style one. Since the old style header reads all of its contents into the global namespace, there is no need for a **namespace** statement.

One other point: For now and for the next few years, you will see many C++ programs that use the old-style headers and that do not include a **namespace** statement. Your C++ compiler will be able to compile them just fine. For new programs, however, you should use the modern style because it is the only style of program that complies with Standard C++. While old-style programs will continue to be supported for many years, they are technically noncompliant.

EXERCISE

1. Before proceeding, try compiling the new-style skeleton program shown above. Although it does nothing, compiling it will tell you if your compiler supports the modern C++ syntax. If it does not accept the new-style headers or the **namespace** statement, substitute the old-style header as described. Remember, if your compiler does not accept new-style code, you must make this change for each program in this book.

1.3 C++ CONSOLE I/O

Since C++ is a superset of C, all elements of the C language are also contained in the C++ language. This implies that all C programs are also C++ programs by default. (Actually, there are some very minor exceptions to this rule, which are discussed later in this book.) Therefore, it is possible to write C++ programs that look just like C programs. While there is nothing wrong with this per se, it does mean that you will not be taking full advantage of C++. To get the maximum benefit from C++, you must write C++ style programs. This means using a coding style and features that are unique to C++.

Perhaps the most common C++ specific feature used by C++ programmers is its approach to console I/O. While you may still use functions such as **printf()** and **scanf()**, C++ provides a new, and better, way to perform these types of I/O operations. In C++, I/O is performed using **I/O operators** instead of I/O functions. The output operator is <<and the input operator is >>. As you know, in C, these are the left and right shift operators, respectively. In C++, they still retain their original meanings (left and right shift) butt they also take on the expanded role of performing input and output. Consider this C++ statement:

```
cout << "This string is output to the screen.\n";
```

This statement causes the string to be displayed on the computer's screen. **cout** is a predefined stream that is automatically linked to the console when a C++ program begins execution. It is similar to C's **stdout**. As in C, C++ console I/O may be redirected, but for the rest of this discussion, it is assumed that the console is being used.

By using the <<output operator, it is possible to output any of C++'s basic types. For example, this statement outputs the value 100.99:

```
cout << 100.99;
```

In general, to output to the console, use this form of the <<operator:

```
cout << expression;
```

Here, *expression* can be any valid C++ expression-including another output expression.
To input a value from the keyboard, use the >>input operator. For example, this fragment inputs an integer value into **num**:

```
int num;
cin >> num;
```

Notice that **num** is *not* preceded by an **&**. As you know, when you use C's **scanf()** function to input values, variables must have their addresses passed to the function so they can receive the values entered by the user. This is not the case when you are using C++'s input operator. (The reason for this will become clear as you learn more about C++.)
In general, to input values from the keyboard, use the form if >>:

```
cin >> variable;
```

Note: *The expanded roles of <<and >>are examples if operator overloading.*
In order to use the C++ I/O operators, you must include the header **<iostream>** in your program. As explained earlier, this is one of C++'s standard headers and is supplied by your C++ compiler.

EXAMPLES

1. This program outputs a string, two integer values, and a double floating-point value:

```
#include <iostream>
using namespace std;

int main()
{
    int i, j;
    double d;

    i = 10;
    j = 20;
    d = 99.101;

    cout << "Here are some values: ";
    cout << i;
    cout << ' ';
    cout << j;
    cout << ' ';
    cout << d;

    return 0;
}
```

The output of this program is shown here.

```
Here are some values:  10 20 99.101
```

Remember: *If you are working with an older compiler, it might not accept the new-style headers and the* ***namespace*** *statements used by this and other programs in this book. If this is the case, substitute the old-style code described in the preceding section.*

2. It is possible to output more than one value in a single I/O expression. For example, this version of the program described in Example 1 shows a more efficient way to code the I/O statements:

```
#include <iostream>
using namespace std;

int main()
{
    int i, j;
    double d;

    i = 10;
    j = 20;
    d = 99.101;

    cout << "Here are some values: ";
    cout << i << ' ' << j << ' ' << d;

    return 0;
}
```

Here the line

```
cout << i << ' ' << j << ' ' << d;
```

outputs several items in one expression. In general, you can use a single statement to output as many items as you like. If this seems confusion, simply remember that the >>output operator behaves like any other C++ operator and can be part of an arbitrarily long expression.

Notice that you must explicitly include spaces between items when needed. If the spaces are left out, the data will run together when displayed on the screen.

3. This program prompts the user for an integer value:

```
#include <iostream>
using namespace std;

int main()
{
    int i;

    cout << "Enter a value: ";
    cin >> i;
    cout << "Here;s your number: | << i << "\n";

    return 0;
}
```

Here is a sample run:

```
Enter a value:  100
```

```
Here's your number:  100
```

As you can see, the value entered by the user is put into **i**.

4. The next program prompts the user for an integer value, a floating-point value, and a string. It then uses one input statement to read all three.

```
#include <iostream>
using namespace std;

int main()
{
     int i;
     float f;
     char s[80];

     cout << "Enter an integer, float, and string: ";
     cin >> i >> f >> s;
     cout << "Here's your data: ";
     cout << i << ' ' << f << ' ' << s;

     return 0;
}
```

As this example illustrates, you can input as many items as you like in one input statement. As in C, individual data items must be separated by whitespace characters (spaces, tabls, or newlines).

When a string is read, input will stop when the first whitespace character is encountered. For example, if you enter the following into the preceding program

```
10 100.12 This is a test
```

the program will display this:

```
10 100.12 This
```

The string is incomplete because the reading of the string stopped with the space after **This**. The remainder of the string is left in the input buffer, awaiting a subsequent input operation. (This is similar to inputting a string by using **scanf()** with the **%s** format.)

5. By default, when you use >>, all input is line buffered. This means that no information is passed to your C++ program until you press ENTER. (In C, the **scanf()** function is line buffered, so this style of input should not be new to you.) To see the effect of line-buffered input, try this program:

```
#include <iostream>
using namespace std;

int main()
{
     char ch;

     cout << "Enter keys, x to stop.\n";
     do
     {
          cout << ": ";
          cin >> ch;
     }
     while(ch!='x');

     return 0;
}
```

When you test this program, you will have to press ENTER after each key you type in order for the corresponding character to be sent to the program.

EXERCISES

1. Write a program that inputs the number of hours that an employee works and the employee's wage. Then display the employee's gross pay. (Be sure to prompt for input.)

2. Write a program that converts feet to inches. Prompt the user for feet and display the equivalent number of inches. Have your program repeat this process until the user enters 0 for the number of feet.

3. Here is a C program. Rewrite it so it uses C++-style I/O statements.

```
#include <stdio.h>

int main(void)
{
    int a, b, d, min;

    printf("Enter two numbers: ");
    scanf("%d%d", &a, &b);
    min = a > b ? b : a;
    for(d = 2; d<min; d++)
        if(((a%d)==0) && ((b%d)==0))
            break;
    if(d==min)
    {
        printf("No common denominators\n");
        return 0;
    }
    printf("The lowest common denominator is %d\n", d);

    return 0;
}
```

1.4 C++ COMMENTS

In C++, you can include comments in your program two different ways. First, you can use the standard, C-like comment mechanism. That is, begin a comment with /* and end it with */. As with C, this type of comment cannot be nested in C++.
The second way that you can add a remark to your C++ program is to use the *single-line comment*. A single-line comment begins with a // and stops at the end of the line. Other than the physical end of the line (that is, a carriage-return/linefeed combination), a single-line comment uses no comment terminator symbol.
Typically, C++ programmers use C-like comments for multiline commentaries and reserve C++-style single-line comments for short remarks.

EXAMPLES

1. Here is a program that contains both C and C++-style comments:

```
/*
    This is a C-like comment.

    This program determines whether
    an integer is odd or even
*/
#include <iostream>
using namespace std;

int main()
{
    int num; // this is a C++ single-line comment

    // read the number
    cout << "Enter number to be tested: ";
    cin >> num;

    // see if even or odd
    if((num%2)==0)
        cout << "Number is even\n";
    else
        cout << "Number is odd\n";

    return 0;
}
```

2. While multiline comments cannot be nested, it is possible to nest a single-line comment within a multiline comment. For example, this is perfectly valid:

```
/*
    This is a multiline comment
    inside of which // is nested a single-line comment.
    Here is the end of the multiline comment.
*/
```

EXERCISES

1. As an experiment, determine whether this comment (which nests a C-like comment within a C++-style, single-line comment) is valid:

```
// This is a strange /* way to do a comment */
```

2. On your own, add comments to the answers to the exercises in Section 1.3.

1.5 CLASSES: A FIRST LOOK

Perhaps the single most important feature of C++ is the *class*. The class is the mechanism that is used to create objects. As such, the class is at the heart of many C++ features. Although the subject of classes is covered in great detail throughout this book, classes are so fundamental to C++ programming that a brief overview is necessary here.
A class is declared using the **class** keyword. The syntax of a **class** declaration is similar to that of a **structure**. Its general form is shown here:

```
class class_name
{
    // private functions and variables
    public:
    // public functions and variables
} object_list;
```

In a class declaration, the *object-list* is optional. As with a structure, you can declare class objects later, as needed. While the *class-name* is also technically optional, from a practical point of view it is virtually always needed. The reason for this is that the *class-name* becomes a new type name that is used to declare objects of the class.
Functions and variables declared inside a class declaration are said to be *members* of that class. By default, all functions and variables declared inside a class are private to that class. This means that they are accessible, only by other members of that class. To declare public class members, the **public** keyword is used, followed by a colon. All functions and variables declared after the **public** specifier are accessible both by other members of the class and by any other part of the program that contains the class.
Here is a simple class declaration:

```
class myclass
{
    // private to myclass
    int a;
    public:
    void set_a(int num);
    int get_a();
};
```

This class has one private variable, called **a**, and two public functions, **set_a()** and **get_a()** Notice that functions are declared within a class using their prototype forms. Functions that are declared to be part of a class are called *member functions*.
Since **a** is private, it is not accessible by any code outside **myclass**. However, since **set_a()** and **get_a()** are members of **myclass**, they can access **a**. Further, **get_a()** and **set_a()** are declared as public members of **myclass** and can be called by any other part of the program that contains **myclass**.
Although the functions **get_a()** and **set_a()** are declared by **myclass**, they are not yet defined. To define a member function, you must link the type name of the class with the name of the function. You do this by preceding the function name with the class name followed by two colons. The two colons are called the *scope resolution operator*. For example, here is the way the member functions **set_a()** and **get_a()** are defined:

```
void myclass::set_a(int num)
{
    a = num;
}

int myclass:: get_a()
{
    return a;
}
```

Notice that both **set_a()** and **get_a()** have access to **a**, which is private to **myclass**. Because **set_a()** and **get_a()** are members of **myclass**, they can directly access its private data.
In general, to define a member function you must use this form:

```
ret_type class_name::func_name(parameter_list)
{
    // body of function
}
```

Here *class-name* is the name of the class to which the function belongs.
The declaration of **myclass** did not define any objects of type **myclass**-it only defines the type of object that will be created when one is actually declared. To create an object, use the class name as a type specifier. For example, this line declares two objects of type **myclass**:

```
myclass ob1, ob2; // these are objects of type myclass
```

Remember: *A class declaration is a logical abstraction that defines a new type. It determines what an object of that type will look like, An object declaration creates a physical entity of that type. That is, an object occupies memory space, but a type definition does not.*
Once an object of a class has been created, your program can reference its public members by using the dot (period) operator in much the same way that structure members are accessed. Assuming the preceding object declaration, the following statement calls **set_a()** for objects **ob1** and **ob2**:

```
ob1.set_a(10); // sets ob1's version of a to 10
ob2.set_a(99); // sets ob2's version of a to 99
```

As the comments indicate, these statements set **ob1**'s copy of **a** to 10 and **ob2**'s copy to 99. Each object contains its own copy of all data declared within the class. This means that **ob1**'s **a** is distinct and different from the **a** linked to **ob2**.
Remember: *Each object of a class has its own copy of every variable declared within the class.*

EXAMPLES

1. As a simple first example, this program demonstrates **myclass**, described in the text. It sets the value of **a** of **ob1** and **ob2** and then displays **a**'s value for each object:

```
#include <iostream>
using namespace std;

class myclass
{
    // private to myclass
    int a;
    public:
    void set_a(int num);
    int get_a();
};

void myclass::set_a(int num)
{
    a = num;
}

int myclass::get_a()
{
    return a;
}
```

```
int main()
{
    myclass ob1, ob2;

    ob1.set_a(10);
    ob2.set_a(99);

    cout << ob1.get_a() << "\n";
    cout << ob2.get_a() << "\n";

    return 0;
```

As you should expect, this program displays the values 10 and 99 on the screen.

2. In **myclass** from the preceding example, **a** is private. This means that only member functions of **myclass** can access it directly. (This is one reason why the public function **get_a()** is required.) If you try to access a private member of a class from some part of your program that is not a member of that class, a compile-time error will result. For example, assuming that **myclass** is defined as shown in the preceding example, the following **main()** function will cause an error:

```
// This fragment contains an error
#include <iostream>
using namespace std;

int main()
{
    myclass ob1, ob2;

    ob1.a = 10; // ERROR! cannot access private member
    ob2.a = 99; // by non-member functions.

    cout << ob1.get_a() << "\n";
    cout << ob2.get_a() << "\n";

    return 0;
}
```

3. Just as there can be public member functions, there can be public member variables as well. For example, if **a** were declared in the public section of **myclass a** could be referenced by any part of the program, as shown here:

```
// This fragment contains an error
#include <iostream>
using namespace std;

class myclass
{
    public:
    // now a is public
    int a;
    // and there is no need for set_a() or get_a()
```

```
};

int main()
{
    myclass ob1, ob2;

    // here a is accessed directly
    ob1.a = 10;
    ob2.a = 99;

    cout << ob1.a << "\n";
    cout << ob2.a << "\n";

    return 0;
}
```

In this example, since **a** is declared as a public member of **myclass**, it is directly accessible from **main()**. Notice how the dot operator is used to access **a**. In general, when you are calling a member function or accessing a member variable from outside its class, the object's name followed by the dot operator followed by the member's name is required to fully specify which object's member you are referring to.

4. To get a taste of the power of objects, let's look at a more practical example. This program creates a class called **stack** that implements a stack that can be used to store characters:

```
#include <iostream>
using namespace std;

#define SIZE 10

// Declare a stack class for characters
class stack
{
    char stck[SIZE]; // holds the stack
    int tos; // index of top of stack

public:
    void init(); // initialize stack
    void push(char ch); // push character on stack
    char pop(); // pop character from stack
};

// Initialize the stack
void stack::init()
{
   tos = 0;
}

// Push a character
void stack::push(char ch)
{
    if(tos==SIZE)
```

```
    {
        cout << "stack is full";
        return;
    }
    stck[tos] = ch;
    tos++;
}

// Pop a character
char stack::pop()
{
   if(tos==0)
   {
       cout << "Stack is empty";
       return 0; // return null on empty stack
    }
    tos--;
    return stck[tos];
}

int main()
{
    stack s1, s2; // create two stacks
    int i;
    // initialize the stacks
    s1.init();
    s2.init();

    s1.push('a');
    s2.push('x');
    s1.push('b');
    s2.push('y');
    s1.push('c');
    s2.push('z');

    for(i=0; i<3; i++)
        cout << "Pop s1: " << s1.pop() << "\n";
    for(i=0; i<3; i++)
        cout << "Pop s2: " << s2.pop() << "\n";

    return 0;
}
```

This program displays the following output:

```
Pop s1: c
Pop s1: b
Pop s1: a
Pop s2: z
Pop s2: y
```

```
Pop s2:  x
```

Let's take a close look at this program now. The class **stack** contains two private variables: **stck** and **tos**. The array **stck** actually holds the characters pushed onto the stack, and **tos** contains the index to the top of the stack. The public stack functions are **init()**, **push()**, and **pop()**, which initialize the stack, push a value, and pop a value, respectively.

Inside **main()**, two stacks, **s1** and **s2**, are created, and three characters are pushed onto each stack. It is important to understand that each stack object is separate from the other. That is, the characters pushed onto **s1** *in no way* affect the characters pushed onto **s2**. Each object contains its own copy of **stck** and **tos**. This concept is fundamental to understanding objects. Although all objects of a class share their member functions, each object creates and maintains *its own data.*

EXERCISES

1. If you have not done so, enter and run the programs shown in the examples for this section.
2. Create a class called **card** that maintains a library card catalog entry. Have the class store a book's title, author, and number of copies on hand. Store the title and author as strings and the number on hand as an integer. Use a public member function called **store()** to store a book's information and a public member function called **show()** to display the information. Include a short **main()** function to demonstrate the class.
3. Create a queue class that maintains a circular queue of integers. Make the queue size 100 integers long. Include a short **main()** function that demonstrates its operation.

1.6 SOME DIFFERENCES BETWEEN C AND C++

Although C++ is a superset of C, there are some small differences between the two, and a few are worth knowing from the start. Before proceeding, let's take time to examine them.

First, in C, when a function takes no parameters, its prototype has the word **void** inside its function parameter list. For example, in C, if a function called **f1()** takes no parameters (and returns a **char**, its prototype will look like this:

```
char f1(void);
```

However, in C++, the **void** is optional. Therefore, in C++, the prototype for **f1()** is usually written like this:

```
char f1();
```

C++ differs from C in the way that an empty parameter list is specified. If the preceding prototype had occurred in a C program, it would simply mean that *nothing is said about* the parameters to the function. In C++, it means that the function *has no parameters.* This is the reason that the preceding examples did not explicitly use **void** to declare an empty parameters list. (The use of **void** to declare an empty parameter list is not illegal; it is just redundant. Since most C++ programmers pursue efficiency with a nearly religious zeal, you will almost never see **void** used in this way.) Remember, in C++, these two declarations are equivalent:

```
char f1();
char f1(void);
```

Another subtle difference between C and C++ is that in a C++ program, all functions must be prototyped. Remember, in C, prototypes are recommended but technically optional. In C++, they are required. As the examples from the previous section show, a member function's prototype contained in a class also serves as its general prototype, and no other separate prototype is required.

A third difference between C and C++ is that in C++, if a function is declared as returning a value, it must return a value. That is, if a function has a return type other than **void**, any **return** statement within that function must contain a value. In C, a non-**void** function is not required to actually return a value. If it doesn't, a garbage value is "returned".
In C, if you don't explicitly specify the return type of a function, an integer return type is assumed. C++ has dropped the "default-to-int" rule. Thus, you must explicitly declare the return type of all functions.
One other difference between C and C++ that you will commonly encounter in C++ programs has to do with where local variables can be declared. In C, local variables can declared only at the start of a block, prior to any "action" statements. In C++, local variables can be declared anywhere. One advantage of this approach is that local variables can be declared close to where they are first used, thus helping to prevent unwanted side effects.
Finally, C++ defines the **bool** data type, which is used to store Boolean (i.e., true/false) values. C++ also defines the keywords **true** and **false**, which are the only values that a value of type **bool** can have. In C++, the outcome of the relational and logical operators is a value of type **bool**, and all conditional statements must evaluate to a **bool** value. Although this might at first seem to be a big change from C, it isn't. In fact, it is virtually transparent. Here's why: As you know, in C, **true** is any nonzero value and **false** is 0. This still holds in C++ because any nonzero value is automatically converted into **false** when used in a Boolean expression. The reverse also occurs: **true** is converted to 1 and **false** is converted to 0 when a **bool** value is used in an integer expression. The addition of **bool** allows more thorough type checking and gives you a way to differentiate between Boolean and integer types. Of course, its use is optional; **bool** is mostly a convenience.

EXAMPLES

1. In a C program, it is common practice to declare **main()** as shown here if it takes no command-line arguments:

   ```
   int main(void)
   ```

   However, in C++, the use of **void** is redundant and unnecessary.

2. This short C++ program will not compile because the function **sum()** is not prototyped:

```
// This program will not compile
#include <iostream>
using namespace std;

int main()
{
   int a, b, c;

   cout << "Enter two numbers: ";
   cin >> a >> b;
   c = sum(a, b);
   cout << "Sum is: " << c;

   return 0;
}

// This function needs a prototype
sum(int a, int b)
{
```

```
    return a+b;
}
```

3. Here is a short program that illustrates how local variables can be declared anywhere within a block:

```
#include <iostream>
using namespace std;

int main()
{
    int i; // local var declared at start of block

    cout << "Enter number: ";
    cin >> i;

    // compute factorial
    int j, fact=1; // vars declared after action statement

    for(j=i; j>=1; j--)
        fact = fact * j;
    cout << "Factorial is " << fact;

    return 0;
}
```

The declaration of **j** and **fact** near the point of first use is of little value in this short example; however, in large functions, the ability to declare variables close to the point of their first use can help clarify your code and prevent unintentional side effects.

4. The following program creates a Boolean variable called **outcome** and assigns it the value **false**. It then uses this variable in an **if** statement.

```
#include <iostream>
using namespace std;

int main()
{
    bool outcome;

    outcome = false;

    if(outcome)
        cout << "true";
    else
        cout << "false";

    return 0;
}
```

As you should expect, the program displays **false**.

EXERCISES

1. The following program will not compile as a C++ program. Why not?

```
// This program has an error.
#include <iostream>
using namespace std;

int main()
{
    f();

    return 0;
}

void f()
{
    cout << "this won't work";
}
```

2. On your own, try declaring local variables at various points in a C++ program. Try the same in a C program, paying attention to which declarations generate errors.

1.7 INTRODUCING FUNCTION OVERLOADING

After classes, perhaps the next most important and pervasive C++ feature is *function overloading*. Not only does function overloading provide the mechanism by which C++ achieves one type of polymorphism, it also forms the basis by which the C++ programming environment can be dynamically extended. Because of the importance of overloading, a brief discussion is given here.

In C++, two or more functions can share the same name as long as either the type of their arguments differs or the number of their arguments differs-or both. When two or more functions share the same name, they are said to be *overloaded.*Overloaded functions can help reduce the complexity of a program by allowing related operations to be referred to by the same name.

It is very easy to overload a function: simply declare and define all required versions. The compiler will automatically select the correct version based upon the number and/or type of the arguments used to call the function.

Note: *It is also possible in C++ to overload operators. However, before you can fully understand operator overloading, you will need to know more about C++.*

EXAMPLES

1. One of the main uses for function overloading is to achieve compile-time polymorphism, which embodies the philosophy of one interface, many methods. As you know, in C programming, it is common to have a number of related functions that differ only by the type of data on which they operate. The classic example of this situation is found in the C standard library. As mentioned earlier in this chapter, the library contains the functions **abs()**, **labs()**, and **fabs()**, which return the absolute value of an integer, a long integer, and a floating-point value, respectively.

   How ever, because three different names are needed due to the three different data types, the situation is more complicated than it needs to be. In all three cases, the absolute value is being returned; only the type of the data differs. In C++, you can correct this

situation by overloading one name for the three types of data, as this example illustrates:

```
#include <iostream>
using namespace std;

// Overload abs() three ways
int abs(int n);
long abs(long n);
double abs(double n);

int main()
{
    cout << "Absolute value of -10: " << abs(-10) << "\n\n";
    cout << "Absolute value of -10L: " << abs(-10L) << "\n\n"
       ;
    cout << "Absolute value of -10.01: " << abs(-10.01) << "\
       n\n";

    return 0;
}

// abs() for ints
int abs(int n)
{
    cout << "In integer abs()\n";
    return n<0 ? -n : n;
}

// abs() for longs
long abs(long n)
{
    cout << "In long abs()\n";
    return n<0 ? -n : n;
}

// abs() for doubles
double abs(double n)
{
    cout << "In double abs()\n";
    return n<0 ? -n : n;
}
```

As you can see, this program defines three functions called **abs()**-one for each data type. Inside **main()**, **abs()** is called using three different types of arguments. The compiler automatically calls the correct version of **abs()** based upon the type of data used as an argument. The program produces the following output:

```
In integer abs()
Absolute value of -10:  10
In long abs()
Absolute value of -10L: 10
```

```
In double abs()
Absolute value of -10.01:  10.01
```

Although this example is quite simple, it still illustrates the value of function overloading. Because a single name can be used to describe a general class of action, the artificial complexity cause by three slightly different names-in this case, **abs()**, **fabs()** and **labs()**-is eliminated. You now must remember only one name-the one that describes the *general* action. It is left to the compiler to choose the appropriate *specific* version of the function (that is, the method) to call. This has the net effect of reducing complexity. Thus, through the use of polymorphism, three names have been reduced to one.

While the use of polymorphism in this example is fairly trivial, you should be able to see how in a very large program, the "one interface, multiple methods" approach can be quite effective.

2. Here is another example of function overloading. In this case, the function **date()** is overloaded to accept the date either as a string or as three integers. In both cases, the function displays the data passed to it.

```
#include <iostream>
using namespace std;

void date(char *date); // date as a string
void date(int month, int day, int year);   //date as numbers

int main()
{
    date("8/23/99");
    date(8, 23, 99);

    return 0;
}

// Date as string.
void date(char *date)
{
    cout << "Date: " << date << "\n";
}

// Date as integers.
void date(int month, int day, int year)
{
    cout << "Date: " << month << "/";
    cout << day << "/" << year << "\n";
}
```

This example illustrates how function overloading can provide the most natural interface to a function. Since it is very common for the date to be represented as either a string or as three integers containing the month, day, and year, you are free to select the most convenient form relative to the situation at hand.

3. So far, you have seen overloaded functions that differ in the data types of their arguments. However, overloaded functions can also differ in the number of arguments, as this example illustrates:

```
#include <iostream>
using namespace std;

void f1(int a);
void f1(int a, int b);

int main()
{
    f1(10);
    f1(10, 20);

    return 0;
}

void f1(int a)
{
    cout << "In f1(int a)\n";
}

void f1(int a, int b)
{
    cout << "In f1(int a, int b)\n";
}
```

4. It is important to understand that the return type alone is not a sufficient difference to allow function overloading. If two functions differ only in the type of data they return, the compiler will not always be able to select the proper one to call. For example, this fragment is incorrect because it is inherently ambiguous.

```
// This is incorrect and will not compile.
int f1(int a);
double f1(int a);
.
.
.
f1(10); // which function does the compiler call???
```

As the comment indicates, the compiler has o way of knowing which version of **f1()** to call.

EXERCISES

1. Create a function called **sroot()** that returns the square root of its argument. Overload **sroot()** three ways: have it return the square root of an integer, a long integer, and a **double**. (To actually compute the square root, you can use the standard library function **sqrt()**.)

2. The C++ standard library contains these three functions:

```
double atof(const char *s);
int atoi(const char *s);
long atol(const char *s);
```

These functions return the numeric value contained in the string pointed to by **s**. Specifically, **atof()** returns a **double**, **atoi()** returns an integer, and **atol()** returns a **long**. Why is it not possible to overload these functions?

3. Create a function called **min()** that returns the smaller of the two numeric arguments used to call the function. Overload **min()** so it accepts characters, integers, and **double**s as arguments.

4. Create a function called **sleep()** that pauses the computer for the number of seconds specified by its single argument. Overload **sleep()** so it can be called with either an integer or a string representation of an integer. For example, both of these calls to **sleep()** will cause the computer to pause for 10 seconds:

```
sleep(10);
sleep("10");
```

Demonstrate that your functions work by including them in a short program. (Feel free to use a delay loop to pause the computer.)

1.8 C++ KEYWORDS

C++ supports all of the keywords defined by C and adds 30 of its own. The entire set of keywords defined by C++ is shown in table below. Also early versions of C++ defined the **overload** keyword, but it is now obsolete.

asm	const_cast	explicit	int	register	switch	union
auto	continue	extern	long	reinterpret_cast	template	unsigned
bool	default	false	mutable	return	this	using
break	delete	float	namespace	short	throw	virtual
case	do	for	new	signed	true	void
catch	double	friend	operator	sizeof	try	volatile
char	dynamic_cast	goto	private	static	typedef	wchar_t
class	else	if	protected	static_cast	typeid	while
const	enum	inline	public	struct	typename	

SKILLS CHECK

Mastery Skills Check

At this point you should be able to perform the following exercises and answer the questions.

1. Give brief descriptions of polymorphism, encapsulation, and inheritance.

2. How can comments be included in a C++ program?

3. Write a program that uses C++-style I/O to input two integers from the keyboard and then displays the result of raising the first to the power of the second. (For example, if the user enters 2 and 4, the result is 24, or 16.)

4. Create a function called **rev_str()** that reverses a string. Overload **rev_str()** so it can be called with either one character array or two. When it is called with one string, have that one string contain the reversal. When it is called with two strings, return the reversed string in the second argument. For example:

```
char s1[80], s2[80];
strcpy(s1, "hello");
rev_str(s1, s2); // reversed string goes in s2, s1 untouched
rev_str(s1); // reversed string is returned in s1
```

5. Given the following new-style C++ program, show how to change it into its old-style form.

```
#include <iostream>
using namespace std;

int f(int a);

int main()
{
    cout << f(10);

    return 0;
}

int f(int a)
{
    return a * 3.1416;
}
```

6. What is the **bool** data type?

This Page Intentionally Left Blank.

(well, except for those 5 words...
and those 6 words...
and those 4 words...
Oh my word, this I'd better curb,
'cause this is so absurd!)